UNIVERSITÉ PARIS-SACLAY M1 MIAGE UE DSID

Projet Big Data

Chaînes d'ingestion et contrôles Bronze–Silver

Kamel Mahdi Gherbi

Année universitaire 2026–2027

Avancement du projet

Le flux 1, consacré à l'API des prix des carburants, a produit un lot brut vérifié dans MinIO. Le chargement PostgreSQL Bronze et les contrôles de la vue Silver ont été validés sur les 9 834 stations de ce lot. Les cinq connexions des trois flux sont actives et la vue d'ensemble Airbyte indique un état sain. Le contrôle Bronze du flux CSV a confirmé 101 codes département distincts. Le contrôle Silver du flux CSV a confirmé le même volume et la présence des champs de traçabilité sur les 101 lignes. Le contrôle Bronze confirme l'arrivée des quatre nouvelles fiches CRM. Le rattachement de l'opportunité au compte est vérifié dans Bronze. Les contrôles de volume, de clés et de traçabilité des trois vues Silver du CRM sont validés.

Table des matières

1 Architecture **3**

1.1 Périmètre et environnement local . . . 3

1.2 Réseau et types de connexion . . . 3

1.3 Parcours des données et synchronisations . . . 3

2 Sources **4**

2.1 Flux 1 — API publique des prix des carburants . . . 4

2.1.1 Choix de la source et contrat de données . . . 4

2.1.2 Test du GET dans PowerShell . . . 4

2.1.3 Configuration et validation dans Airbyte Builder . . . 5

2.2 Flux 2 — Fichier CSV . . . 5

2.2.1 Choix du fichier et contrat de données . . . 5

2.2.2 Contrôle du fichier brut . . . 5

2.3 Flux 3 — EspoCRM et MariaDB . . . 6

2.3.1 Périmètre de la source . . . 6

3 Bronze **6**

3.1 Flux 1 — De MinIO à PostgreSQL . . . 6

3.1.1 Lot brut contrôlé dans MinIO . . . 6

3.1.2 Lecture S3 et configuration PostgreSQL . . . 6

3.1.3 Incident de chargement et correction . . . 7

3.1.4 Structure observée de la table Bronze . . . 7

3.2 Flux 2 — Chargement du CSV . . . 7

3.3 Flux 3 — Chargement du CRM . . . 8

4 Silver **8**

4.1 Règles communes . . . 8

4.2 Flux 1 — Vue des stations et prix des carburants . . . 8

4.2.1 Grain et clé . . . 8

4.2.2 Dictionnaire des colonnes et conversions . . . 9

4.2.3 Dédoublonnage et réconciliation . . . 9

4.3 Flux 2 — Données du CSV . . . 9

4.4 Flux 3 — Entités CRM . . . 10

5 Tests **10**

5.1 Flux 1 — Contrôles du chargement Bronze . . . 10

5.2 Flux 1 — Contrôle préalable des conversions . . . 11

5.3 Flux 1 — Résultats des huit contrôles Silver . . . 11

5.4 Flux 2 — Contrôles de la vue Silver 11
5.5 Flux 3 — Contrôles des vues Silver 11

6 Delta CRM **12**
6.1 État initial et nouvelles saisies 12
6.2 Vérification de l'arrivée en Bronze 12

7 Docker et Airbyte **13**
7.1 Installation locale et réseau 13
7.2 Inventaire de l'environnement exécuté 13
7.2.1 Images Docker et version de l'outil de déploiement 13
7.2.2 Identification de MinIO 13
7.3 Qualification de l'environnement de référence 13
7.4 Incident de publication dans Builder 14
7.4.1 Diagnostic 14
7.4.2 Correction et résultat 14
7.5 Connexions et conventions de nommage 14
7.5.1 Connexion API vers MinIO 14
7.5.2 Chemin d'écriture et lecture S3 14
7.5.3 Noms techniques et noms affichés 14

8 Reproductibilité **15**
8.1 Ordre de reprise du flux API 15
8.2 Vérification de la reprise et livrables 15

1 Architecture

1.1 Périmètre et environnement local

Le projet a été exécuté sur un poste Windows avec Docker Desktop 4.92.0, Docker Engine 29.8.0 et Docker Compose v5.5.1. Docker Desktop exécute les conteneurs en `linux/amd64`. Docker Compose démarre les services du kit sur le réseau `dsid-bigdata-network`. Airbyte est déployé séparément dans un cluster Kubernetes local `kind`, créé avec `abctl v0.30.4`. Le chart Helm et l'application Airbyte sont tous deux en version 2.5.0. Les sorties d'inventaire figurent à la section 7.

Composant	Version ou référence	Rôle et accès local
Airbyte	2.5.0	Exécute les synchronisations. Interface Web sur le port 8000.
MinIO	Image construite par le kit	Stockage objet compatible S3. API sur 9000, console sur 9001. La référence de construction et la version du binaire actif figurent à la section 7.
PostgreSQL	18.6	Base `warehouse` sur 5432. `bronze` reçoit les données chargées et `silver` expose les vues SQL standardisées.
MariaDB	11.8.9	Base opérationnelle `espocrm` sur 3306.
EspoCRM et daemon	10.0.8	Saisie des fiches et exécution des tâches du CRM. Interface Web sur 8081.
Adminer	6.1.0	Interface de consultation des bases et d'exécution des contrôles SQL sur 8082.

1.2 Réseau et types de connexion

Le navigateur du poste accède aux interfaces Web via `localhost`. Les connecteurs Airbyte s'exécutent dans le cluster `kind`, sur un réseau distinct de Compose. Pour joindre les ports publiés sur la machine hôte, ils utilisent l'adresse `host.docker.internal`.

Liaison	Protocole et configuration utilisés
Sources publiques	Requête HTTPS GET sans clé pour l'API carburants. Lecture HTTPS du CSV Insee avec le connecteur File.
MinIO	Connexion S3 compatible via `http://host.docker.internal:9000`. Le bucket `dsid-raw` est privé. Les clés d'accès sont configurées localement dans Airbyte. La lecture a été validée avec Source S3 4.16.4.
PostgreSQL	Connecteur Postgres 3.0.24, hôte `host.docker.internal`, port 5432, base `warehouse`, schéma cible `bronze`.
MariaDB	Connecteur MySQL, hôte `host.docker.internal`, port 3306, base `espocrm`. Pour la reprise, le kit crée le compte dédié `airbyte` avec un droit de lecture `SELECT`.

1.3 Parcours des données et synchronisations

Flux	Parcours des données
1. API carburants	API publique → Airbyte (connecteur Builder) → MinIO RAW → Airbyte Source S3 → PostgreSQL Bronze
2. Fichier CSV	CSV Insee → Airbyte File Source → MinIO RAW → Airbyte Source S3 → PostgreSQL Bronze
3. EspoCRM	EspoCRM → MariaDB → Airbyte MySQL Source → PostgreSQL Bronze
Couche Silver	PostgreSQL Bronze → scripts SQL par source → PostgreSQL Silver

Les cinq connexions Airbyte utilisent un déclenchement manuel. L'API et le CSV ont chacun deux connexions, l'une pour déposer les données dans MinIO et l'autre pour les charger en Bronze. Le CRM utilise une connexion directe de MariaDB à PostgreSQL. Pour l'API, le mode *Full Refresh – Overwrite* relit le périmètre sélectionné et maintient un état courant. Le manifeste identifie le lot d'origine par sa clé objet et son empreinte SHA-256.

Vue des cinq connexions Airbyte. Elle montre les sources, les destinations et le déclenchement manuel.

2 Sources

2.1 Flux 1 — API publique des prix des carburants

2.1.1 Choix de la source et contrat de données

Le jeu *Prix des carburants en France, flux instantané v2* est publié par la DGCCRF. L'API Explore v2.1 expose les données en JSON par requête HTTPS GET, sans clé. Le jeu est publié sous Licence Ouverte 2.0 avec attribution de la source.

Chaque objet du tableau `results` représente une station dans l'état observé pendant la collecte. Le champ `id` identifie la station. Le lot identifie une collecte et permet de distinguer les observations successives d'une même station.

Groupe	Exemples de champs et de types observés
Identification	`id` numérique dans le JSON, code postal `cp`, ville et adresse en texte.
Prix	`gazole_prix` et prix des autres carburants numériques. Les champs `*_maj` sont des chaînes horodatées.
Disponibilité	Tableaux de carburants disponibles ou indisponibles et champs décrivant les ruptures publiées.

Console API du ministère Jeu de données et licence

2.1.2 Test du GET dans PowerShell

Le filtre `where=startswith(cp, '75')` a limité le test aux stations dont le code postal commence par 75. Le tri explicite `order_by=id` facilitait la comparaison des pages. La taille de 20 a permis de contrôler deux changements de page et l'arrêt sur une dernière page partielle.

PowerShell

```
HTTP 200 | offset=0  | count=20 | total_count=47
HTTP 200 | offset=20 | count=20 | total_count=47
HTTP 200 | offset=40 | count=7  | total_count=47

Stations recues         : 47 / 47
Identifiants distincts  : 47
Identifiants absents    : 0
Doublons                : 0
Stations hors perimetre : 0
Validation              : True
```

Capture du test PowerShell. Elle montre les réponses HTTP, les volumes par page et les contrôles des identifiants et du périmètre.

2.1.3 Configuration et validation dans Airbyte Builder

Le connecteur utilise la route nationale `/records`.

Réglage	Choix et justification
Méthode et accès	GET sans authentification pour interroger la route publique.
Flux	`prix_carburants_france` dans le YAML Builder versionné. Le flux S3 qui relit le lot s'appelle `prix_carburants`.
Sélecteur	`results`, tableau qui contient les stations.
Clé	`id`, identifiant métier de la station.
Pagination	*Offset Increment*. `limit` indique la taille de page et `offset` indique le nombre d'enregistrements à sauter.

Le test Builder sur Paris a vérifié l'extraction de `results` et le passage entre les trois pages. Pour la collecte nationale, le filtre et le tri explicite du test ont été retirés. L'absence de `select` et de transformation Builder conserve les champs et les valeurs transmis par la source.

La taille de page est passée de 20 à 100 pour réduire le nombre de requêtes. L'aperçu national a confirmé les décalages 100 et 200. La documentation de la route fixe un plafond de 10 000 pour `offset + limit`. La complétude se vérifie en comparant le nombre collecté au `total_count` annoncé pendant le même job. Les relevés du projet ne permettent pas d'établir le quota de débit applicable à cette route.

Capture du test Builder. Elle montre la configuration et l'aperçu du flux de test. Pagination du flux national. Cette capture montre le réglage à 100 enregistrements par page et l'aperçu Builder. Documentation de la pagination

2.2 Flux 2 — Fichier CSV

2.2.1 Choix du fichier et contrat de données

Le jeu retenu est le fichier des départements du Code officiel géographique 2026 publié par l'Insee. Une ligne décrit un département, identifié par le code `DEP`. Ce code est traité comme du texte afin de préserver les zéros initiaux et les codes alphanumériques. Le CSV comporte un en-tête, un séparateur virgule et un encodage UTF-8. Le fichier original contient 101 enregistrements et 101 codes distincts. L'Insee le diffuse sous Licence Ouverte. Une copie du fichier est fournie sous `data/v_departement_2026.csv`.

Fiche du jeu de données et licence

Fichier CSV source de l'Insee

2.2.2 Contrôle du fichier brut

Le connecteur File lit le CSV via HTTPS. Airbyte écrit la sortie datée dans le bucket `dsid-raw`, sous `raw/csv/departements`. La synchronisation du 13 novembre 2026 a chargé 101 enregistrements dans MinIO. Après téléchargement de la copie archivée de l'original, le contrôle PowerShell a confirmé que son SHA-256 correspond à celui du fichier officiel. Le CSV généré par Airbyte a une empreinte différente. Le manifeste consigne les empreintes, la clé de l'objet généré et le nombre d'enregistrements chargés.

- Synchronisation CSV vers MinIO Airbyte indique 101 enregistrements chargés.
- Objet CSV dans MinIO La console affiche l'objet daté produit par Airbyte.
- Contrôle SHA-256 de l'original archivé Le résultat PowerShell `True` confirme l'égalité des empreintes.

— Manifeste du lot CSV.

2.3 Flux 3 — EspoCRM et MariaDB

2.3.1 Périmètre de la source

La source MySQL `mariadb_espocrm` lit la base MariaDB `espocrm` et réplique les flux `account`, `contact` et `opportunity` vers PostgreSQL Bronze. Chaque ligne représente une fiche de son entité et la clé métier est le champ `id`. Les colonnes `contact.account_id` et `opportunity.account_id` portent les rattachements au compte vérifiés dans ce scénario.

Une capture de configuration affiche l'utilisateur `espocrm`. Le compte utilisé par le job réussi n'est pas établi par les preuves du dépôt. La procédure de reprise utilise le compte MariaDB dédié `airbyte` du kit et vérifie son droit `SELECT` avant le test de connexion.

3 Bronze

3.1 Flux 1 — De MinIO à PostgreSQL

3.1.1 Lot brut contrôlé dans MinIO

Le job Airbyte `5` a chargé 9 834 enregistrements. MinIO affiche un fichier JSONL de 25,4 MiB dans le bucket privé `dsid-raw`.

```
raw/api/prix_carburants/extraction_date=2026-11-13/1794580622068_0.jsonl
```

Le fichier téléchargé contient 9 834 lignes, soit le volume chargé affiché dans Airbyte. Le `total_count` national du job n'est pas consigné. La concordance des volumes établit la cohérence du lot collecté et chargé.

PowerShell

```
Lignes JSONL : 9834
Job Airbyte lu dans _airbyte_meta.sync_id : 5
SHA-256 :
AD91F416382A02FCEA107849390F3AF73449B5BF95AB29027CB9D9870214C80C
```

Synchronisation API vers MinIO. La chronologie Airbyte montre une exécution réussie avec 9 834 enregistrements chargés.

Objet JSONL dans MinIO. La console affiche la clé datée et la taille de 25,4 MiB dans le bucket `dsid-raw`.

Manifeste du lot. Il associe le job, la clé MinIO, le volume et l'empreinte du fichier.

3.1.2 Lecture S3 et configuration PostgreSQL

La source `minio_raw_prix_carburants` lit le bucket `dsid-raw` en JSONL avec le motif suivant.

```
raw/api/prix_carburants/extraction_date=*/*.jsonl
```

Les caractères génériques acceptent les dates et les noms de fichiers des prochains lots. Le manifeste conserve la clé exacte pour rejouer une collecte précise.

La connexion `minio_raw` → `bronze` utilise le schéma de destination `bronze` et le préfixe de flux `api_`. Le mode *Full Refresh – Overwrite* recharge la table à partir des objets sélectionnés par le motif S3. Le déclenchement manuel a permis de vérifier le premier chargement.

3.1.3 Incident de chargement et correction

Les contrôles de connexion MinIO et PostgreSQL ont permis de retrouver l'objet et d'atteindre le schéma cible. La réplication échouait ensuite à la création de la table. Le journal de la destination indiquait `column "_airbyte_raw_id" specified more than once`.

Le JSONL contient les métadonnées du job API vers MinIO. Le connecteur PostgreSQL génère ses propres colonnes techniques au chargement. Pour éviter les collisions, quatre champs entrants ont été décochés dans le schéma du flux `prix_carburants`.

— Champs décochés : `_airbyte_raw_id`, `_airbyte_extracted_at`, `_airbyte_meta` et `_airbyte_generation_id`.
— Champs conservés : `_airbyte_data`, `_ab_source_file_url` et `_ab_source_file_last_modified`.

La synchronisation a réussi après cette correction. Le JSONL dans MinIO conserve les métadonnées du job 5. PostgreSQL contient les métadonnées générées lors de son propre chargement et les deux champs de provenance du fichier.

Les contrôles PostgreSQL établissent le résultat du transfert MinIO vers Bronze. Le numéro 5 identifie le job API vers MinIO. L'identifiant du job MinIO vers PostgreSQL ne figure pas dans les preuves disponibles.

3.1.4 Structure observée de la table Bronze

La table `bronze.api_prix_carburants` contient sept colonnes.

Colonne	Type PostgreSQL
`_airbyte_raw_id`	`character varying`
`_airbyte_extracted_at`	`timestamp with time zone`
`_airbyte_meta`	`jsonb`
`_airbyte_generation_id`	`bigint`
`_airbyte_data`	`jsonb`
`_ab_source_file_url`	`character varying`
`_ab_source_file_last_modified`	`character varying`

L'inventaire de `_airbyte_data` a identifié 47 champs métier. Leurs présences varient selon les stations. L'identifiant `id` est accessible dans cet objet JSON.

Structure de la table Bronze. La requête affiche les sept colonnes, les types PostgreSQL et les champs de provenance.

3.2 Flux 2 — Chargement du CSV

La connexion *minio_raw vers bronze*, identifiée par la source `minio_raw_departements`, figure parmi les cinq connexions saines. La configuration documentée cible `bronze.csv_departements`. Le motif S3 retient les sorties CSV datées et exclut le dossier `original/`. Le contrôle SQL dans la base `warehouse` a confirmé 101 lignes dans `bronze.csv_departements`. Ce volume correspond au nombre d'enregistrements du fichier officiel et au nombre chargé par Airbyte vers MinIO. Le test compte 101 codes département distincts, sans valeur absente ou vide et sans doublon. Les codes

`01`, `2A` et `2B` apparaissent chacun une fois, ce qui confirme la conservation du zéro initial et des codes alphanumériques vérifiés.

Requête et résultat du contrôle Bronze du CSV.

3.3 Flux 3 — Chargement du CRM

La connexion *espocrm vers bronze* apparaît saine dans Airbyte. L'inventaire SQL de la base `warehouse` confirme trois tables dans le schéma `bronze`. Les volumes observés correspondent au relevé initial de la source MariaDB.

Table PostgreSQL	MariaDB	Bronze
`bronze.espocrm_account`	1	1
`bronze.espocrm_contact`	2	2
`bronze.espocrm_opportunity`	1	1

Inventaire des tables CRM dans Bronze Comptage des lignes CRM dans Bronze.

4 Silver

4.1 Règles communes

Cinq vues PostgreSQL versionnées sous `sql/silver/` exposent une ligne par clé métier dans chaque entité. Elles calculent la standardisation à partir de l'état courant de Bronze à chaque requête. Après une synchronisation, les mêmes règles s'appliquent immédiatement aux données chargées. Les colonnes suivent le *snake_case*. Le système source, l'identifiant source et l'horodatage d'ingestion assurent la traçabilité.

Dans les cinq vues, `ingested_at_utc` reprend directement `_airbyte_extracted_at`, de type `timestamptz`. PostgreSQL conserve l'instant et l'affiche dans le fuseau de la session. Le paramètre SQL `SET TIME ZONE 'UTC'` permet de l'afficher en UTC. Les champs métier `created_at` et `modified_at` du CRM gardent leur type source `timestamp without time zone`. Leur conversion en instants UTC nécessite de confirmer le fuseau source.

Le dédoublonnage traite les occurrences répétées d'une même clé. Une clé vide est écartée. Pour une clé présente plusieurs fois, la vue retient l'extraction la plus récente et utilise l'identifiant technique Airbyte pour départager les égalités.

4.2 Flux 1 — Vue des stations et prix des carburants

4.2.1 Grain et clé

La vue `silver.api_station`, définie dans `sql/silver/api_station.sql`, reprend le modèle fourni. Le grain est une ligne par station dans l'état courant chargé en Bronze. `record_id` et `source_record_id` reprennent l'identifiant `id` de l'API en texte.

4.2.2 Dictionnaire des colonnes et conversions

Groupe	Colonnes et règles appliquées
Clés	`record_id` et `source_record_id` en `text`, issus de `id`.
Adresse	`code_postal`, `ville`, `adresse`, `code_departement` et `code_region` en `text`. Le code postal provient de `cp`.
Prix	`gazole_prix`, `e10_prix`, `e85_prix`, `gplc_prix`, `sp95_prix` et `sp98_prix` en `numeric(12,3)`.
Horodatages métier	Les six champs source `*_maj` deviennent `*_mis_a_jour_at` en `timestamptz`.
Disponibilité	`carburants_disponibles` et `carburants_indisponibles` en `jsonb`.
Traçabilité	`source_system` en `text`, `ingested_at_utc` en `timestamptz` et `source_file_key` en `character varying`.

Les espaces en bordure des champs textuels sont retirés. Une chaîne vide après nettoyage représente une valeur manquante et devient `NULL`. Le type texte conserve les zéros initiaux, comme dans le code postal `01000`. Le type `numeric(12,3)` conserve la précision observée sur les prix, par exemple `2.301`. Le décimal exact convient aux prix exprimés au millième. Le type `double precision` utilise une représentation binaire approchée qui peut introduire des écarts lors des comparaisons ou des calculs. Un prix absent reste `NULL`.

L'horodatage `ingested_at_utc` reprend `_airbyte_extracted_at` en Bronze. Les horodatages métier proviennent des champs de mise à jour de l'API. La valeur de `source_system` est `api_prix_carburants` et `source_file_key` reprend la référence de l'objet MinIO.

La vue expose les champs nécessaires à l'identification des stations, à leur localisation administrative et à l'analyse des prix et de la disponibilité. Les autres champs de l'API, dont les horaires, les services et le détail des ruptures, restent consultables en Bronze.

4.2.3 Dédoublonnage et réconciliation

La vue écarte les identifiants absents ou vides. Pour chaque identifiant renseigné, `ROW_NUMBER()` classe les lignes par horodatage d'ingestion décroissant. L'identifiant technique `_airbyte_raw_id` départage les égalités. La vue conserve la ligne de rang 1. Sur le lot contrôlé, aucune ligne n'a été écartée.

4.3 Flux 2 — Données du CSV

La vue `silver.csv_departement` présente une ligne par code département. Elle standardise les noms des sept champs métier en `snake_case`, supprime les espaces en bordure des valeurs et convertit les chaînes vides en valeurs nulles. Les codes sont conservés en texte pour préserver les zéros initiaux et les codes alphanumériques.

Le code département fournit `record_id` et `source_record_id`. La vue conserve également le système source, l'horodatage d'extraction Airbyte et la référence du fichier MinIO. Les lignes sans code sont écartées. Pour un même code, la vue conserve la ligne dont l'horodatage d'extraction est le plus récent. En cas d'égalité, l'identifiant technique Airbyte départage les lignes par ordre décroissant. Le contrôle Bronze a relevé 101 codes uniques et aucune valeur absente.

Le script de création est `sql/silver/csv_departement.sql`.

4.4 Flux 3 — Entités CRM

Les vues `silver.espocrm_account`, `silver.espocrm_contact` et `silver.espocrm_opportunity` présentent une ligne par identifiant source et par entité. Les champs textuels sélectionnés sont normalisés par suppression des espaces en bordure et conversion des chaînes vides en valeurs nulles. Les vues conservent le statut de suppression logique, les clés de liaison disponibles et les champs de traçabilité du template. Le montant conserve son type numérique et sa devise, et la date de clôture conserve son type `date`. Les vues conservent les clés de liaison en texte. Les contacts et les opportunités référencent le compte par `account_id`, rapproché de `record_id` dans la vue des comptes. Ce choix préserve le grain de chaque entité et permet de vérifier séparément la présence de la fiche et celle du compte attendu.

Les lignes sans identifiant sont écartées. Pour chaque identifiant, la vue retient l'extraction Airbyte la plus récente et départage les égalités par ordre décroissant de l'identifiant technique Airbyte. Les trois scripts de création se trouvent dans `sql/silver/`.

5 Tests

5.1 Flux 1 — Contrôles du chargement Bronze

Contrôle	Attendu	Observé
Nombre de lignes dans Bronze	9 834	9 834
Identifiants de station distincts	9 834	9 834
Identifiants absents ou vides	0	0
Occurrences en doublon	0	0
Nombre de références de fichier distinctes	1	1

Les 9 834 lignes référencent la clé MinIO `raw/api/prix_carburants/extraction_date=2026-11-13/1794580622068_0.jsonl`. Le comptage concorde avec celui du fichier téléchargé.

Volume et clés de Bronze. Les 9 834 identifiants sont tous présents et distincts.

Provenance du lot Bronze. Les 9 834 lignes référencent un seul objet JSONL.

```
WITH stations AS (
    SELECT NULLIF(TRIM(_airbyte_data ->> 'id'), '') AS id_station
    FROM bronze.api_prix_carburants
)
SELECT COUNT(*) AS lignes_chargees,
       COUNT(DISTINCT id_station) AS ids_distincts,
       COUNT(*) FILTER (WHERE id_station IS NULL) AS ids_absents,
       COUNT(id_station) - COUNT(DISTINCT id_station)
           AS occurrences_en_doublon
FROM stations;

SELECT column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'bronze'
  AND table_name = 'api_prix_carburants'
ORDER BY ordinal_position;

SELECT _ab_source_file_url AS fichier_source,
       COUNT(*) AS nombre_lignes
FROM bronze.api_prix_carburants
GROUP BY _ab_source_file_url;
```

5.2 Flux 1 — Contrôle préalable des conversions

Le test a vérifié la conversion en `numeric(12,3)` de tous les prix renseignés et la validité des horodatages renseignés. Les valeurs absentes restent manquantes.

Carburant	Prix renseignés	Prix invalides ou arrondis	Horodatages invalides
E10	7 336	0	0
E85	3 762	0	0
Gazole	9 319	0	0
GPLc	1 504	0	0
SP95	2 740	0	0
SP98	7 135	0	0

Contrôle préalable des conversions. Pour les six carburants, la requête relève zéro prix invalide ou arrondi et zéro horodatage invalide.

5.3 Flux 1 — Résultats des huit contrôles Silver

Contrôle	Résultat observé et interprétation
Clé non nulle	0 clé absente en Silver.
Unicité	9 834 clés distinctes pour 9 834 lignes.
Doublons	0 occurrence en doublon en Bronze et en Silver sur ce lot.
Nommage	Les 24 noms sont conformes au *snake_case*.
Types	Les 24 types correspondent au dictionnaire Silver. Les six prix ont une précision de 12 et une échelle de 3.
Réconciliation	9 834 lignes Bronze et 9 834 lignes Silver. Le test de réconciliation retourne vrai.
Traçabilité	0 ligne sans traçabilité complète selon le contrôle du système, de l'identifiant, de l'horodatage et du fichier source.
Règle métier	0 station avec un prix renseigné négatif.

Les requêtes de bilan et d'inspection des colonnes figurent dans `sql/tests/api_station.sql`. Les résultats portent sur le lot contrôlé.

Bilan des contrôles Silver. Le résultat confirme la réconciliation des 9 834 lignes, l'unicité des clés, la traçabilité et l'absence de prix négatif.

Nommage et types des colonnes Silver. L'inventaire confirme 24 noms conformes et le type `numeric(12,3)` des six prix.

5.4 Flux 2 — Contrôles de la vue Silver

Le contrôle exécuté dans PostgreSQL retourne 101 lignes pour 101 identifiants distincts, comme en Bronze. Il relève zéro identifiant absent, zéro occurrence en doublon et zéro ligne sans les informations de traçabilité attendues. Les codes `01`, `2A` et `2B` apparaissent chacun une fois.

Le contrôle est reproductible avec `sql/tests/csv_departement.sql`. Requête et résultat du contrôle Silver du C

5.5 Flux 3 — Contrôles des vues Silver

La requête `sql/tests/espocrm.sql` confirme les volumes du lot Bronze après le test du delta. Le nombre d'identifiants distincts correspond au nombre de lignes pour chaque entité.

Entité	Bronze	Silver	Identifiants distincts
Comptes	2	2	2
Contacts	4	4	4
Opportunités	2	2	2

Pour chacune des trois vues, le contrôle relève zéro identifiant absent, zéro doublon et zéro ligne présentant une anomalie sur le système source, l'identifiant source ou l'horodatage d'ingestion.

Résultats des contrôles Silver du CRM.

Le contrôle des rattachements, inclus dans le même script, retrouve les deux contacts et l'opportunité créés pour le test du delta. Pour les trois fiches, le compte référencé et le compte retrouvé dans la vue Silver correspondent au compte de test `6af78794aba6cf05e`.

Contrôle des trois rattachements au compte dans Silver.

L'inventaire des trois vues confirme que les 37 noms de colonnes respectent le format `snake_case`. Les identifiants et clés de liaison sont de type `text`. L'horodatage d'ingestion est de type `timestamp with time zone`. Les champs de création et de modification conservent leur type source `timestamp without time zone`. Le montant de l'opportunité est de type `numeric`, la devise de type `text`, la probabilité de type `bigint` et la date de clôture de type `date`. Les indicateurs de suppression et d'opposition aux appels sont de type `boolean`.

Inventaire des noms et types des colonnes CRM dans Silver.

6 Delta CRM

6.1 État initial et nouvelles saisies

Les relevés initiaux dans MariaDB et PostgreSQL Bronze montrent un compte, deux contacts et une opportunité. Le test consiste à créer un nouveau compte, deux contacts et une opportunité dans EspoCRM, puis à rechercher leurs identifiants dans Bronze après synchronisation.

6.2 Vérification de l'arrivée en Bronze

Entité	Avant	Après	Écart
Comptes	1	2	+1
Contacts	2	4	+2
Opportunités	1	2	+1

La requête de contrôle recherche les quatre identifiants EspoCRM dans les tables correspondantes. Elle relève exactement une occurrence par identifiant et confirme les écarts attendus.

État des trois flux CRM dans Airbyte. L'écran indique deux comptes, quatre contacts et deux opportunités chargés lors de la dernière synchronisation affichée.

Requête et résultat du contrôle des quatre identifiants CRM.

Le contrôle de la relation entre l'opportunité et le compte joint `bronze.espocrm_opportunity` à `bronze.espocrm_account` sur `opportunity.account_id` et `account.id`. La requête retourne une ligne et confirme que l'opportunité `6af787ec38e9debdd` référence le compte de test `6af78794aba6cf05e`.

Contrôle du rattachement de l'opportunité au compte dans Bronze.

7 Docker et Airbyte

7.1 Installation locale et réseau

Sur le poste Windows, Docker Desktop 4.92.0, Docker Engine 29.8.0 et Docker Compose v5.5.1 ont été observés. Airbyte 2.5.0 est déployé avec `abctl`. La stack a été démarrée depuis `compose/` après création du fichier local `.env` à partir de `.env.example`.

Airbyte est accessible sur le port 8000. MinIO expose son API sur 9000 et sa console sur 9001. PostgreSQL utilise 5432, MariaDB 3306, EspoCRM 8081 et Adminer 8082. Depuis Airbyte sur Windows, `host.docker.internal` permet de joindre les services exposés sur la machine hôte. Le kit décrit les variantes macOS et Linux.

Procédure Docker du dépôt

7.2 Inventaire de l'environnement exécuté

La validation de la configuration avec `docker compose config -quiet` n'a affiché aucune erreur. Les six services Compose étaient démarrés. PostgreSQL, MariaDB et EspoCRM affichaient l'état `healthy`. Le contrôle `abctl local status` indiquait `deployed` pour Airbyte, avec le chart Helm 2.5.0 et l'application 2.5.0. Le contrôleur `ingress-nginx` utilisait le chart 4.15.1 et l'application 1.15.1.

7.2.1 Images Docker et version de l'outil de déploiement

La sortie de `docker compose images` associe chaque conteneur à l'image effectivement utilisée. Les colonnes sont resserrées pour tenir sur la page.

CONTAINER	REPOSITORY	TAG	PLATFORM	IMAGE ID	SIZE	CREATED
dsid-adminer	adminer	6.1.0	linux/amd64	8df93d7bb431	164MB	7 weeks ago
dsid-espocrm	espocrm/espocrm	10.0.8	linux/amd64	817f2b063de2	1.08GB	2 months ago
dsid-espocrm-daemon	espocrm/espocrm	10.0.8	linux/amd64	817f2b063de2	1.08GB	2 months ago
dsid-mariadb	mariadb	11.8.9	linux/amd64	c4124d1afa87	453MB	2 weeks ago
dsid-minio	<none>	<none>	unknown	2d3964653720	0B	N/A
dsid-postgres	postgres	18.6	linux/amd64	e3d24f6906d0	644MB	8 days ago

La sortie de `abctl version` identifie l'outil utilisé pour déployer Airbyte.

```
version: v0.30.4
```

7.2.2 Identification de MinIO

Le kit configure la construction depuis la référence `RELEASE.2025-10-15T17-29-55Z`. L'image active `2d3964653720` est affichée sans tag. La commande `docker exec dsid-minio minio -version` identifie le binaire actif comme `DEVELOPMENT.GOGET`, compilé avec Go 1.24.13 pour `linux/amd64`. Le tag de construction et l'identification du binaire sont relevés séparément.

7.3 Qualification de l'environnement de référence

Les relevés Docker et Airbyte documentent la stack en fonctionnement sur le poste Windows. Source S3 4.16.4 accède au lot API dans MinIO via `http://host.docker.internal:9000`. Les contrôles SQL documentent les chargements Bronze et les résultats Silver des trois sources.

7.4 Incident de publication dans Builder

7.4.1 Diagnostic

Le test Builder réussissait. L'action *Publish* renvoyait une erreur HTTP 500. Le journal serveur consulté avec `kubectl logs` signalait l'absence de référence d'image pour les manifestes de version majeure 7. Le YAML déclarait la version `7.28.2`.

7.4.2 Correction et résultat

1. Vérifier que le pod PostgreSQL interne `airbyte-db-0` est en cours d'exécution.
2. Vérifier l'empreinte SHA-256 de l'image `airbyte/source-declarative-manifest:7.28.2` avec `docker buildx imagetools inspect`.
3. Ajouter la référence de la version majeure 7 dans la table `declarative_manifest_image_version` de `db-airbyte`, avec `ON CONFLICT DO NOTHING`.
4. Relire l'entrée et relancer la publication.

PowerShell a affiché `INSERT 0 1`, puis la version `7.28.2` associée à la version majeure 7. La publication a réussi avec le même manifeste YAML.

Ticket Airbyte et contournement documenté

7.5 Connexions et conventions de nommage

7.5.1 Connexion API vers MinIO

La connexion affichée dans Airbyte est `api_prix_carburants` → `minio_raw`. Elle relie *API Prix des carburants France* à *MinIO RAW Prix des carburants*. Les 47 champs détectés ont été sélectionnés. Le préfixe de flux `api_` indique l'origine des données.

Le déclenchement manuel permet de contrôler les exécutions. Le mode *Full Refresh – Overwrite* relit le périmètre national et maintient le dernier état dans la zone d'écriture. Pour conserver un lot, archiver ses fichiers et son manifeste hors de ce préfixe avant une nouvelle synchronisation.

7.5.2 Chemin d'écriture et lecture S3

JSONL conserve un objet par ligne et les structures imbriquées. Le chemin de base est `raw/api/prix_carburants`. Le modèle de chemin ajoute le dossier daté `extraction_date=YYYY-MM-DD`. Le chemin de base identifie déjà le jeu, ce qui permet d'omettre le nom du flux dans le modèle. Le job 5 a confirmé le chemin obtenu.

La lecture a été effectuée avec Source S3 4.16.4 et la destination PostgreSQL 3.0.24. La source S3 utilise le bucket privé `dsid-raw`, l'endpoint `http://host.docker.internal:9000` et la région `us-east-1`. Elle réplique les enregistrements JSONL. Les identifiants sont saisis dans les champs privés d'Airbyte.

7.5.3 Noms techniques et noms affichés

Les noms techniques utilisent des minuscules et des underscores. Le modèle `<origine>_to_<destination>` indique le sens du transfert lorsqu'un nouveau nom technique est choisi. Le rapport reprend les noms effectivement affichés dans Airbyte pour relier les connexions aux journaux et aux preuves. Dans PostgreSQL, le préfixe `api_` identifie l'origine de la table Bronze.

8 Reproductibilité

8.1 Ordre de reprise du flux API

1. Démarrer la stack du kit et vérifier les accès à Airbyte, MinIO et PostgreSQL.
2. Importer le manifeste YAML dans Builder, tester le flux et publier le connecteur si nécessaire.
3. Créer la source API et la connexion vers MinIO. Exécuter la collecte et relever le job et la clé de l'objet.
4. Contrôler le volume et le SHA-256 du JSONL. Enregistrer les résultats dans le manifeste du lot.
5. Exécuter la connexion Source S3 vers PostgreSQL Bronze. Conserver les trois champs entrants retenus et vérifier la structure, le volume, les clés et la provenance.
6. Vérifier les conversions des six carburants, exécuter `sql/silver/api_station.sql`, puis les requêtes de `sql/tests/api_station.sql`.

Le dépôt fournit le manifeste du connecteur sous `airbyte/api_prix_des_carburants_france_v_2.yaml`. Le motif S3 est réutilisable pour les collectes suivantes. Pour rejouer exactement un lot, utiliser sa clé et vérifier son SHA-256 à partir du manifeste. Le mode *Overwrite* recharge Bronze avec les objets sélectionnés. La vue Silver reflète ensuite le contenu courant de Bronze.

8.2 Vérification de la reprise et livrables

Le guide de reprise décrit l'ordre de démarrage, la configuration des cinq connexions, puis l'exécution des vues et contrôles SQL. Le guide Airbyte précise les paramètres observés et ceux à vérifier.

Le démarrage et les trois chargements sont documentés sur le poste Windows de référence. La vérification du dossier publié couvre les chemins des guides, les scripts et les annexes. Les preuves disponibles ne couvrent pas un rejeu intégral à partir d'une installation neuve. La version exacte de la destination S3 n'est pas établie par ces relevés.

Pour le CSV, le guide décrit l'archivage de l'original, le contrôle SHA-256 et le chargement de l'objet produit par Airbyte. Pour le CRM, il décrit le contrôle initial, la création des fiches et la vérification de leurs identifiants après synchronisation. Une nouvelle collecte API produit un lot distinct dont le volume peut varier.

Le dossier technique accompagne le PDF. Il contient le README, les fichiers Compose, `.env.example`, les configurations Airbyte sans secrets, les scripts SQL, les manifestes et les preuves associées aux résultats.